



Desarrollo de un Sensor de Colores por Reflexión

Beck, Federico

Departamento de Ingeniería Electrónica,
Facultad Regional San Francisco, Córdoba, Argentina.
federicobeck91@gmail.com

Perez, Mariano

Departamento de Ingeniería Electrónica,
Facultad Regional San Francisco, Córdoba, Argentina.
perezmariano7401@gmail.com



Índice

1. Introducción	3
1.1. Antecedentes	4
1.2. Objetivos	4
2. Desarrollo	4
2.1. Descripción del Dispositivo	4
2.2. Circuito de Acondicionamiento	7
2.3. Caracterización del LDR	8
2.4. Proceso de Medición	13
2.5. Implementación en Microcontrolador	14
2.5.1. Protocolo de Comunicación	14
2.5.2. Conversor Analógico-Digital	15
2.5.3. División de Tareas	16
2.6. Aplicación de Línea de Comandos	18
2.6.1. Módulo <code>serialpal.py</code>	18
2.6.2. Módulo <code>cli.py</code>	19
2.6.3. Scripts Auxiliares	20
3. Resultados	20

1. Introducción

El color es una percepción visual que surge debido a la detección de diferentes longitudes de onda que no son absorbidas por un objeto o superficie. El conjunto de señales de distintas frecuencias compone el color de dicho objeto. Éstas, dentro del espectro visible de la luz, abarcan una gama de colores que va desde el rojo, con frecuencias más bajas y longitudes de onda más largas, hasta el violeta, con frecuencias más altas y longitudes de onda más cortas.

Durante el desarrollo de este trabajo, se diseñó e implementó un dispositivo que permite obtener un estimado del color de una superficie mediante la reflexión de luz. Este utiliza una resistencia dependiente de la luz (LDR) para la detección de la luz reflejada desde la superficie objetivo.

El principio de funcionamiento consiste en colocar el dispositivo sobre la superficie que se desea medir, y los iluminantes del sensor iluminan la superficie utilizando luz roja, verde, y azul por separado. Una vez iluminada la superficie, se mide la resistencia del LDR, y se estima la intensidad de la luz reflejada. El proceso se repite para cada color, y al final se obtiene una estimación del color de la superficie mediante un código RGB de 24 bits, el cual es un formato común y ampliamente utilizado para representar color, en especial de forma digital.

Debido a la baja trazabilidad de los LDR en general y su amplio rango de respuesta (generalmente desde unos cuantos kilo-ohmios hasta el orden de los mega-ohmios), que además varía según la longitud de onda de la luz incidente, se implementó un procedimiento para caracterizar la respuesta del LDR. El objetivo es determinar la respuesta del LDR a las distintas longitudes de onda para mejorar las mediciones obtenidas.

Para tomar las mediciones se desarrolló un circuito de acondicionamiento para el LDR y la placa de desarrollo Raspberry Pi Pico, que utiliza el microcontrolador RP2040. El microcontrolador a su vez implementa un protocolo de comunicación simple mediante USB (utilizando la clase de comunicación puerto serie virtual), que le permite funcionar como dispositivo USB e intercambiar mensajes con una PC u otro host USB.

Por último, se desarrolló una aplicación CLI (interfaz de línea de comandos), que permite la comunicación con el dispositivo a través de USB. La principal motivación fue para facilitar el análisis de datos y la caracterización del LDR utilizando lenguajes de programación de alto nivel, tales como Python.



1.1. Antecedentes

Las opciones comerciales de este tipo de dispositivos suelen medir color al incidir sobre la superficie objetivo, rayos provenientes de una fuente de luz blanca. El color se determina mediante un arreglo de foto-detectores, que generalmente consisten de foto-diodos con filtros RGB; cada foto-diodo detecta la intensidad de luz reflejada para una longitud de onda específica, y se determina el color de la superficie en base a las intensidades relativas de luz captadas por cada uno. Este trabajo busca explorar la posibilidad de cumplir la misma función utilizando componentes más fácilmente accesibles y económicos, compensando las desventajas evidentes del LDR mediante una secuencia de calibración que permita obtener resultados repetibles.

1.2. Objetivos

El objetivo de este trabajo consiste en desarrollar un dispositivo de bajo costo que permita determinar el color de un objeto con un grado de precisión aceptable mediante la reflexión de luz sobre su superficie. A su vez, se implementará un procedimiento de calibración para conocer la respuesta del LDR utilizado en el dispositivo. Para facilitar la operación y el procesamiento de las mediciones, se desarrollará además una aplicación de escritorio que permita interactuar con el dispositivo y visualizar los resultados obtenidos.

2. Desarrollo

A continuación se describen en detalle los distintos aspectos del dispositivo, incluyendo el hardware utilizado y el circuito de acondicionamiento del LDR, así como la implementación del firmware del microcontrolador y la aplicación CLI.

Todo el contenido desarrollado en el presente informe, incluyendo el código de fuente del microcontrolador, aplicación de escritorio y esquemáticos y simulaciones de los circuitos se pueden encontrar en el repositorio de GitHub github.com/marianop9/reflective-color-sensor.

2.1. Descripción del Dispositivo

El dispositivo en su forma más simple consiste de un LDR que actúa como transductor, y un iluminante que hace incidir luz de color rojo, verde y azul sobre la superficie objetivo. Dado que el LDR varía su resistencia en base a la cantidad de luz que incide sobre su superficie, es necesario convertir este cambio de resistencia en una tensión que pueda ser muestreada por un

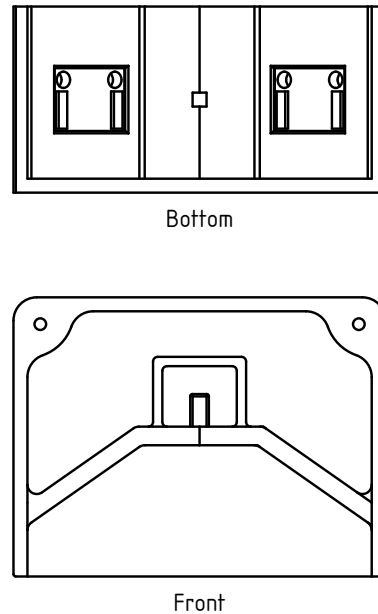


Figura 1: Vistas de la carcasa diseñada para el dispositivo.

convertor analógico-digital (ADC). Para ello se diseñó un circuito de adaptación, descrito en detalle en el Apartado 2.2, que convierte la resistencia del LDR en tensión, y es capaz de funcionar sobre un amplio rango de valores de resistencia.

Para poder tomar mediciones es necesario aislar el dispositivo de la luz ambiente, ya que esta interferiría con los iluminantes propios del dispositivo y distorsionaría las mediciones. Por ello, se diseñó una carcasa especializada donde los iluminantes y el LDR pueden funcionar sin interferencia de la luz externa. Las vistas frontal e inferior se muestran en la Figura 1.

La misma cuenta con dos compartimientos internos. El compartimiento inferior se apoya sobre la superficie objetivo y es iluminado por los LEDs; en la vista inferior de la Figura 1 se observa las ranuras donde van colocados los dos LEDs utilizados. El LDR se coloca en el compartimiento central, que se conecta con el compartimiento inferior mediante una pequeña apertura, lo que permite que la luz reflejada incida directamente sobre la superficie del LDR. Por último, el compartimiento superior se utiliza para las conexiones eléctricas de todos los componentes, y cuenta con una salida al exterior de la carcasa donde estos se conectan al circuito de acondicionamiento y micro-controlador.



Como se mencionó anteriormente, el dispositivo está basado en la placa de desarrollo Raspberry Pi Pico, que utiliza el microcontrolador RP2040 [4]. Este cuenta con dos núcleos ARM Cortex-M0+, ADC de 12 bits, e implementa la capa física del protocolo USB, que permite su funcionamiento como dispositivo USB. El microcontrolador cuenta con más que suficiente poder de procesamiento para esta aplicación, pero su elección se basa principalmente en su disponibilidad y bajo costo, la facilidad de programación, y la capacidad de implementar interfaces de hardware específicos, cuya utilidad se discute a continuación.

El sensor utiliza LEDs RGB WS2812 como emisores, ya que cuentan con un factor de forma compacto, intensidad regulable, y un protocolo de comunicación sencillo. Este protocolo se basa en una señal cuadrada, donde la longitud de los pulsos altos y bajos definen un 0 lógico o 1 lógico. De esta forma se envía un código RGB de 24 bits que define el color de luz emitido por cada LED.

Pese a la simpleza del protocolo de comunicación, este funciona a una frecuencia elevada, de alrededor de 800 kHz. Las implementaciones típicas de drivers para estos dispositivos suelen consistir de procedimientos de ‘bit-banging’ altamente optimizados, normalmente escritos directamente en lenguaje ensamblador. Sin embargo, el microcontrolador RP2040 incluye hardware especializado para desarrollar interfaces de hardware especializados, denominado I/O programable (PIO) [4].

Las capacidades PIO del RP2040 permiten crear sub-programas que no requieren tiempo de procesador y pueden controlar los puertos GPIO del microcontrolador de manera precisa y confiable, lo que lo hace ideal para la implementación de un driver para los LEDs WS2812.

Durante el desarrollo se probaron varios LDR de sulfuro de cadmio (CdS) en distintas condiciones de iluminación y se comprobó que todos tienen respuestas distintas ante los mismos estímulos de luz. Sin embargo, se pudo establecer que toman valores de resistencia en el rango de $1\text{ k}\Omega$ a $1000\text{ k}\Omega$. Además, tal como lo indica la hoja de datos del LDR GL5528 [2] utilizada como referencia, y la Figura 2, un LDR típico tiene distintas sensibilidades según la longitud de onda de la luz incidente.

Por estos motivos, se decidió desarrollar un procedimiento de calibración, que se debe realizar antes de tomar mediciones. Otros aspecto a mencionar de los LDR es que su respuesta no es lineal, sino que su resistencia disminuye exponencialmente con la luminancia, es decir, con la cantidad de luz que incide sobre la superficie sensible del LDR. La respuesta típica del LDR puede aproximar con la Ecuación (1).

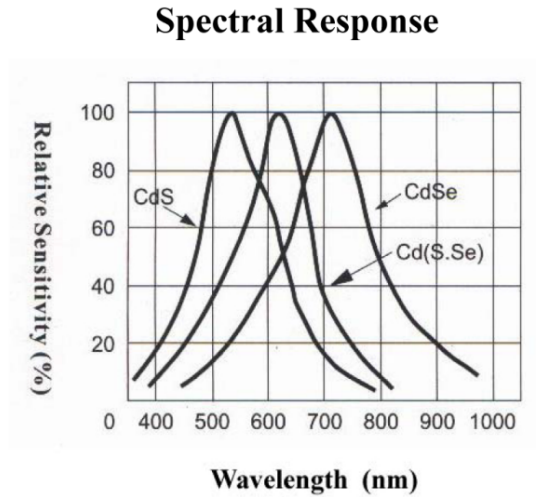


Figura 2: Respuesta espectral del LDR GL5528 según su hoja de datos.

$$R = K_a E^{-\gamma} \quad (1)$$

2.2. Circuito de Acondicionamiento

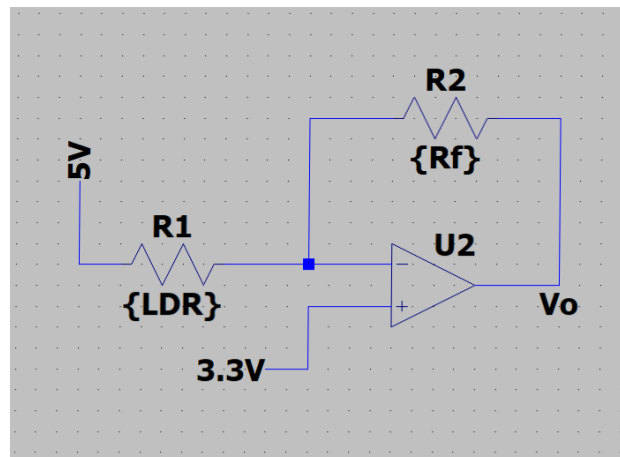


Figura 3: Esquemático del circuito de acondicionamiento.

Un LDR disminuye su resistencia conforme aumenta la cantidad de luz que incide sobre su superficie sensible, es decir, la iluminancia. Para poder medir estos cambios de resistencia y poder determinar cuanta luz se refleja



desde la superficie objetivo al LDR, se construyó el circuito de la Figura 3. Este consiste de un amplificador inversor con una tensión de offset fija en su terminal no inversor.

La tensión V_o a la salida del amplificador operacional de la Figura 3 se puede expresar como:

$$V_o = V_+ + (V_+ - V_s) \frac{R_f}{R_{LDR}} \quad (2)$$

, donde V_+ es la tensión de referencia en la entrada no inversora (3,3 V), V_s es la tensión a la que se conecta una de las terminales del LDR (5 V), y R_f y R_{LDR} corresponden a la resistencia de realimentación y el LDR, respectivamente.

La tensión de referencia V_+ se eligió de manera tal que coincida con la tensión de referencia del ADC, que en el caso del RP2040 es de 3,3 V[4]. De esta forma, se protege al ADC ya que la tensión de salida del circuito de acondicionamiento nunca superará este valor, y a la vez, permite a todo el dispositivo funcionar con una única fuente de alimentación de 5 V.

Además, debido al amplio rango de valores de resistencia que puede tomar el LDR, se utilizaron distintos valores de resistencia R_f que permiten ajustar la ganancia del circuito de acondicionamiento. De lo contrario, utilizando una resistencia R_f fija, el circuito solo funcionaría dentro de un rango limitado de la respuesta del LDR, y quedaría saturado para valores de resistencia fuera de este rango.

Por este motivo, el circuito de adaptación utiliza tres valores de resistencia R_f discretos, 1 k Ω , 10 k Ω y 100 k Ω , que aseguran que la respuesta del circuito no se sature en el rango de funcionamiento del LDR previamente establecido (1 k Ω a 1000 k Ω). La Figura 4 muestra la respuesta del circuito para los distintos valores de resistencia R_f .

Se utilizó el switch bilateral cuádruple CD4066 [1] para permitir el control programable de la resistencia R_f . Este circuito cuenta con cuatro canales independientes y está ampliamente disponible.

2.3. Caracterización del LDR

Para poder tomar mediciones confiables y repetibles utilizando un LDR, resulta necesario un proceso de calibración, ya que no todos se comportan de la misma forma. Además, de la Figura 2 se puede concluir que cada LDR se exhibirá respuestas distintas según la longitud de onda de la luz incidente.

Por estos motivos, en el desarrollo de este trabajo se buscó poder caracterizar el LDR del dispositivo, calculando los parámetros K_a y γ de la

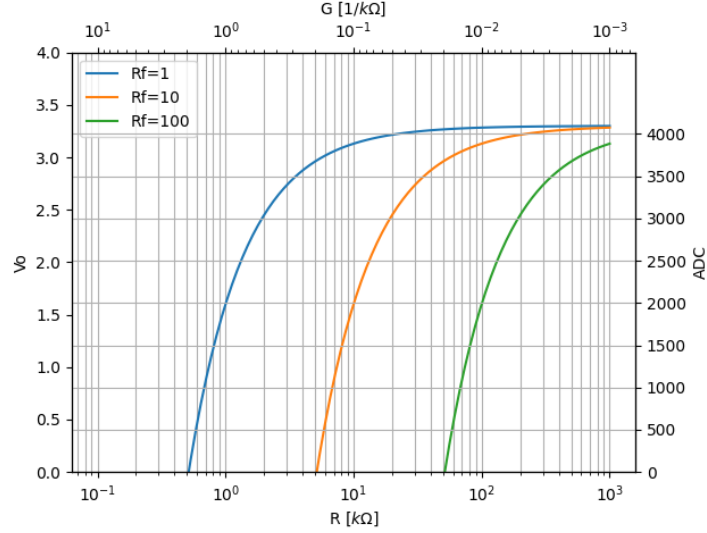


Figura 4: Tensión de salida del circuito de acondicionamiento en función de la resistencia del LDR.

Ecuación (1) de manera empírica.

Otro detalle a mencionar es que en el presente trabajo se decidió trabajar en términos de la conductancia G del LDR, en lugar de su resistencia R . Esto se debe principalmente a que de esta manera, la conductancia aumenta al aumentar la cantidad de luz que incide sobre el LDR. Por lo tanto, las Ecuaciones (1) y (2) se pueden expresar en términos de la conductancia como:

$$G = K_a E^\gamma \quad (3)$$

$$V_o = V_+ + (V_+ - V_s) R_f G \quad (4)$$

De la Ecuación (3) se puede despejar fácilmente una ecuación para la conductancia del LDR en función de la tensión de salida del circuito de acondicionamiento:

$$G = \frac{V_+ - V_o}{R_f * (V_s - V_+)} \quad (5)$$

, donde V_+ es la tensión de referencia del ADC (3,3 V), V_s es la tensión de alimentación del circuito (5 V), y R_f es la resistencia de realimentación.

El procedimiento de caracterización consiste en colocar el dispositivo sobre una superficie de calibración y configurar los LED para iluminar la su-



perficie a distintas intensidades, tomando muestras de la tensión en la salida del circuito de acondicionamiento para cada una.

Una vez que se recolectan suficientes muestras, se aplica la técnica de mínimos cuadrados para encontrar los parámetros de Ecuación (3) que mejor se aproximen a los resultados obtenidos.

La superficie de calibración se eligió de manera arbitraria, priorizando superficies de color blanco y opacas, tal que se minimice la reflexión especular en favor de la reflexión difusa.

Como se mencionó previamente, y en base a lo observado en la Figura 2, el LDR no responde de la misma manera a todas las longitudes de onda. Por lo tanto, este barrido de intensidades de los LED se realizó una vez para cada color utilizado por el sensor: rojo, verde y azul.

Además, debido a que no se cuentan con los elementos necesarios para medir la cantidad de luz que incide sobre la superficie sensible del LDR, es decir, la iluminancia E , para este proceso de caracterización se aproximó la iluminancia E como la intensidad c configurada para los LEDs del dispositivo.

La intensidad c de cada LED del dispositivo se puede configurar mediante un código RGB de 24 bits, utilizando 8 bits para cada canal. Esto significa que cada LED se puede configurar para estar apagado, o en uno de 255 niveles de intensidad.

Reemplazando la aproximación $E \approx c$ en la Ecuación (3), se obtiene:

$$G = K_a c^\gamma \quad (6)$$

Para la secuencia de barrido, se eligieron valores de intensidad desde 0 a 255 con espaciado logarítmico. Se realizaron pruebas utilizando 30 y 40 valores, y se obtuvieron resultados similares. Para cada paso del barrido se registró la intensidad configurada para los LED, la tensión muestreada por el ADC 400 ms luego de ser configurado el valor de intensidad, y el valor de la resistencia R_f .

Además, se definió una región de funcionamiento donde el dispositivo no se considera saturado. Los límites de esta región se eligieron en base a un análisis de la tensión muestreada en el ADC de 12 bits del RP2040. Se encontró que para muestras del ADC con códigos por debajo de 850, no se registraban cambios al variar la intensidad de los LED. Lo mismo ocurrió para muestras con códigos por encima de 3800. Por lo tanto, cuando una muestra se encuentra fuera de este rango, se ajusta la resistencia R_f para cambiar la ganancia del circuito, aumentando o reduciendo la tensión de salida según sea necesario, y se repite la medición.

El barrido siempre comienza utilizando el valor de resistencia más grande, $R_f = 100 \text{ k}\Omega$. Esto se debe a que para intensidades de luz bajas, la conductan-



cia del LDR es muy pequeña, por lo tanto, según la Ecuación (3) se necesita una resistencia R_f grande para obtener un valor de tensión que se encuentre dentro del rango de funcionamiento del sensor.

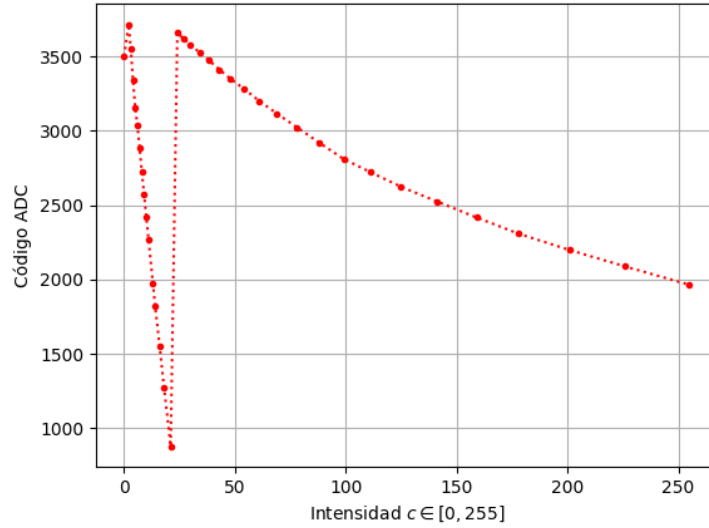


Figura 5: Muestras recolectadas en un barrido de 40 valores de intensidad para el canal rojo.

En la Figura 5 se observa las muestras obtenidas para un barrido del canal rojo. Las discontinuidades corresponden a los cambios de la resistencia R_f . Se observa además que las muestras se encuentran siempre dentro de la región de funcionamiento definida anteriormente (850 a 3800 cuentas del ADC).

En la Figura 6 se muestra el resultado de aplicar la Ecuación (5) a las muestras de la Figura 5. Se utiliza una escala logarítmica para la intensidad, mostrada en el eje x , ya que la respuesta del LDR no es lineal, tal como lo demuestra la Ecuación (6).

Una vez recolectadas las muestras y calculada la conductancia del LDR para cada una, se procedió aplicando la técnica de mínimos cuadrados para obtener una aproximación de la respuesta dispositivo. La i -ésima muestra recolectada se expresa como:

$$G_i = K_a c_i^\gamma$$

Dado que se quieren encontrar los valores de K_a y γ , se aplica la función logaritmo de ambos lados y se obtiene:

$$\ln G_i = \ln K_a + \gamma \ln c \quad (7)$$

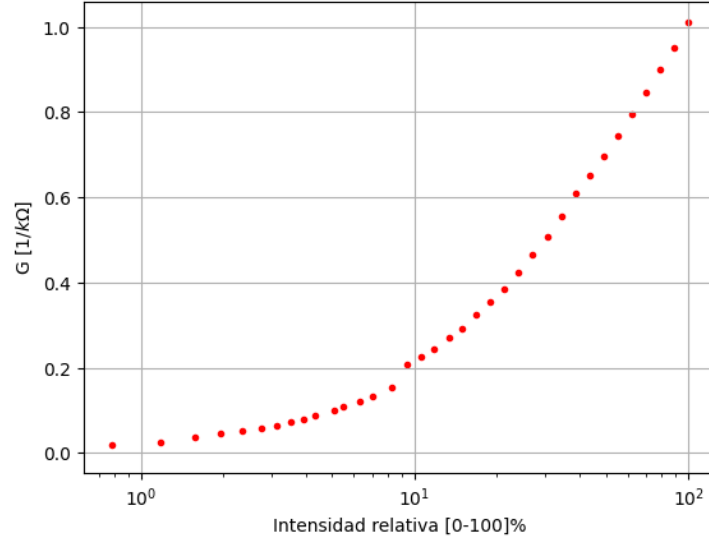


Figura 6: Conductancia calculada a partir del barrido de intensidad para el canal rojo.

Considerando N muestras distintas, se arma un sistema de ecuaciones y se expresa en forma de matrices:

$$\begin{bmatrix} \ln G_1 \\ \ln G_2 \\ \vdots \\ \ln G_N \end{bmatrix} = \begin{bmatrix} 1 & \ln c_1 \\ 1 & \ln c_2 \\ \vdots & \vdots \\ 1 & \ln c_N \end{bmatrix} \begin{bmatrix} \ln K_a \\ \gamma \end{bmatrix} \quad (8)$$

El valor de la matriz de las variables desconocidas se puede obtener a partir de la Ecuación (9), donde Y es la matriz de las conductancias, A es la matriz de los coeficientes (1 y $\ln c_i$), y x la matriz de las variables que se quieren calcular.

$$x = (A^T A)^{-1} A^T Y \quad (9)$$

La aplicación de escritorio que se utilizó para interactuar con el dispositivo cuenta con funcionalidad para exportar las muestras recolectadas en formato JSON, lo cual se detalla a fondo en la Apartado 2.6. Por lo tanto, se utilizaron las librerías de Python *pandas* y *numpy* para analizar los datos recolectados.

En la Figura 7 se muestra la curva de la Ecuación (6) obtenida con los parámetros K_a y γ calculados en base a los resultados obtenidos para el color rojo. Se observa que la aproximación obtenida es muy buena para valores de intensidad bajos, pero se vuelve menos precisa para valores mas altos.

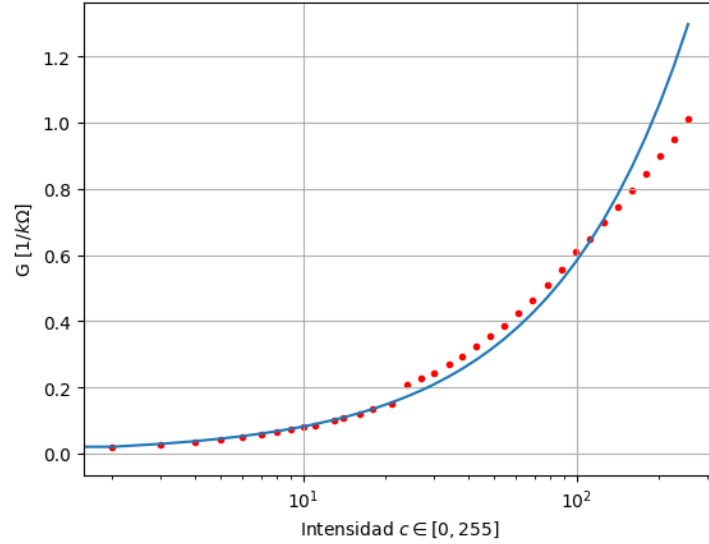


Figura 7: Comparación de la conductancia calculada del barrido de intensidad para el canal rojo, y la aproximación encontrada para la Ecuación (6).

2.4. Proceso de Medición

Una vez que se calibró el dispositivo y se encontraron los valores de K_a y γ para cada color, ya es posible realizar mediciones. El proceso de medición comienza colocando el dispositivo sobre la superficie que se desea medir. Luego, el dispositivo enciende sus LED a la máxima intensidad utilizando los colores rojo, verde y azul por separado. Se toman muestras de la tensión a la salida del circuito de acondicionamiento para cada color, y se calcula la conductancia G del LDR que se obtuvo para cada uno.

Luego, se utilizan los parámetros obtenidos durante la calibración para determinar el color de la superficie. Debido a que la intensidad c de la Ecuación (6) se mide en el rango de 0 a 255, los resultados para cada color se pueden combinar y representar directamente utilizando el código de colores RGB.

Además, al igual que durante el proceso de calibración, se evita tomar resultados saturados asegurando que el dispositivo trabaje dentro de la región de funcionamiento previamente definida. En caso de que una muestra arroje un resultado fuera de esta zona, se ajusta el valor de la resistencia R_f (si es posible), y se repite la medición.



2.5. Implementación en Microcontrolador

El microcontrolador RP2040 es el encargado de controlar los periféricos y coordinar todas las operaciones del dispositivo, y actúa como dispositivo USB que permite al dispositivo conectarse a una PC y comunicarse mediante un protocolo de comunicación por puerto serie simple.

La implementación del firmware en el microcontrolador se realizó en el lenguaje de programación C, utilizando la capa de abstracción de hardware *Pico-SDK* [3] provista por el fabricante.

Para funcionar como dispositivo USB, se utilizó la librería *TinyUSB* [5], la cual ofrece una implementación del stack USB para dispositivos embebidos.

Además, se utilizó el sistema operativo de tiempo real *FreeRTOS*. La principal motivación es que este brinda herramientas para lograr la sincronización de las distintas tareas que lleva a cabo el dispositivo, lo cual es útil debido a la naturaleza asíncrona de la comunicación serial.

El microcontrolador a su vez es el encargado de manejar todos los periféricos del dispositivo, los cuales incluyen:

- Conversor analógico-digital (ADC)
- LED WS2812
- Switch bilateral cuádruple CD4066
- LED indicador de estado

Se implementó un driver para cada uno de estos, permitiendo a la aplicación principal utilizarlos sin conocer sus detalles de implementación.

2.5.1. Protocolo de Comunicación

Se implementó un protocolo de comunicación simple para la comunicación dispositivo-PC. El mismo se basa en una versión modificada de la librería *esadigov/ascii-communication-protocol*, disponible en GitHub.

El protocolo implementado envía mensajes de texto plano, denominados comandos, desde la PC al dispositivo. Estos comandos son interpretados por el dispositivo para ejecutar alguna de sus funciones. Todos los comandos siguen el siguiente formato:

`&<nombre-comando>,[argumento1],[argumento2]*`

Cada comando tiene un nombre único, y opcionalmente puede enviar hasta dos argumentos adicionales. Por ejemplo, el comando para encender ambos LED del dispositivo con color verde se envía como:



```
&SET_LED,0,0x00ff00*
```

Una vez que el dispositivo termina de ejecutar la operación, el mismo arma un mensaje de respuesta para enviar de vuelta a la PC. Debido a que la respuesta muchas veces consiste de valores numéricos, como pueden ser una serie de muestras del ADC, esta respuesta se puede enviar utilizando una combinación de texto plano y datos binarios, utilizando el siguiente formato:

```
&<tipo-respuesta>,<cantidad-bytes>;<bytes-mensaje>
```

A diferencia del formato de un comando, el mensaje de respuesta no puede utilizar un terminador ASCII como *, ya que el mensaje de respuesta podría incluir este mismo carácter. Además, si la respuesta contiene datos binarios, existe la posibilidad de que alguno de los bytes de la respuesta tenga el mismo valor ASCII que el carácter terminador. Por este motivo, el formato de respuesta especifica primero el número de bytes del mensaje enviado, permitiendo que la aplicación de PC pueda leer el mensaje completo.

2.5.2. Conversor Analógico-Digital

Además de un ADC de 12 bits, el RP2040 cuenta además con un controlador DMA que puede configurarse para trabajar junto al ADC. Esto permite al microcontrolador tomar una gran cantidad de muestras y guardarlas en memoria sin requerir de la intervención del procesador.

Pese a que el dispositivo no supone una carga importante para el procesador del RP2040, y que no se toma un número significativo de muestras del ADC, se decidió utilizarlo de todas formas ya que es muy fácil de configurar y reduce y simplifica el código al no tener que escribir lógica para tomar y guardar muestras individuales.

El ADC se configuró con una frecuencia de muestreo de 500 Hz, ya que el transductor LDR se caracteriza por tener una respuesta lenta. Sin embargo, se probaron distintos valores de frecuencia y se obtuvieron resultados similares.

Al iniciar el dispositivo, se inicializa el ADC por código y se conecta a un canal DMA libre. El DMA se configura para leer desde el registro de muestras del ADC (`adc_hw->fifo`) [4], y escribir cada muestra en un arreglo en memoria. El tamaño del arreglo utilizado fue de 40 muestras.

Debido a que la frecuencia de DMA es mucho mayor que la frecuencia de muestreo del ADC, se conecta la señal de solicitud de datos (`DREQ`) del ADC al canal DMA, de manera que el DMA solo realice una transferencia cada vez que el ADC toma una nueva muestra.

Finalmente, dado que la toma de muestras se realiza de manera asíncrona mediante DMA, es necesario sincronizar el final de la transferencia para que



el programa pueda continuar. Por lo tanto, se configuró una interrupción para indicar el final de la transferencia por parte del DMA.

2.5.3. División de Tareas

Las tareas que realiza el microcontrolador se pueden separar en dos partes clave: la comunicación USB por medio de la cual se reciben los comandos que debe ejecutar, y la ejecución de dichos comandos. Debido a que cada una tiene lógica específica, ya sea la decodificación de comandos y codificación de respuestas, la lógica propia del stack USB, o la lógica individual que requiere la ejecución de cada comando, se definieron dos tareas: `usb_task()` y `worker_task()`.

La estructura de la tarea `usb_task()` se muestra en el Listado 1. La misma debe cumplir tres tareas específicas:

1. Ejecutar el stack USB mediante la función `tud_task()` de TinyUSB.
2. Recibir mensajes de respuesta a enviar por USB hacia el host.
3. Enviar las respuestas al host.

Un aspecto crítico de la librería TinyUSB es que la misma no se ejecuta automáticamente como un proceso adicional de la aplicación, sino que se debe asegurar su ejecución llamando a la función `tud_task()` de manera regular. De lo contrario, no se procesarían los eventos del stack USB y el sensor dejaría de ser detectado como dispositivo USB.

Listado 1: Estructura de la tarea encargada de la comunicación por USB.

```
void usb_task(void *arg) {  
    while (1) {  
        tud_task();  
  
        if (!msg_in_buffer) {  
            size_t n = xMessageBufferReceive(tx_msg_buffer, ...);  
            ...  
        }  
  
        if (msg_in_buffer) {  
            ...  
            tud_cdc_write(...);  
            ...  
        }  
    }  
}
```




A su vez, la tarea `usb_task()` también es la encargada de recibir los mensajes provenientes desde el host, e interpretarlos como comandos. Esto se logra utilizando la función callback provista por TinyUSB: `tud_cdc_rx_cb()`, cuya lógica de decodificación se muestra en el Listado 2. Cuando se reciben datos, esta función es ejecutada por el stack USB cuando corre `tud_task()`. Cada mensaje recibido se interpreta como comando y envía por medio de una cola a la tarea `worker_task()`.

Listado 2: Callback para recibir e interpretar comandos recibidos desde el host.

```
void tud_cdc_rx_cb(uint8_t itf) {
    uint8_t rx_buf[ACP_CMD_MAX_SIZE + 1];

    uint32_t n = tud_cdc_read(rx_buf, ACP_CMD_MAX_SIZE);
    if (n == 0)
        return;

    acp_command_t cmd;
    bool result = acp_parse_command(&cmd, rx_buf, n);
    if (result) {
        xQueueSend(cmd_queue, &cmd, 0);
    } else {
        // error...
    }
}
```

Por otro lado, la segunda tarea `worker_task()` es responsable de recibir y ejecutar los comandos, y devolver las respuestas a la tarea `usb_task()` para enviarlas al host. Su estructura general se muestra en el Listado 3.

Esto se logra recibiendo los comandos a través de una cola de FreeRTOS. La tarea bloquea hasta recibir un nuevo comando, y luego ejecuta su correspondiente acción según el tipo de comando recibido. Una vez terminado, cada comando arma su mensaje de respuesta, y lo envía a la `usb_task()` mediante un buffer de mensajes.

La estructura general de la tarea `usb_task()` se muestra a continuación:

Listado 3: Estructura de la tarea procesadora.

```
void worker_task(void *arg) {
    acp_command_t cmd;
    while (1) {
        xQueueReceive(cmd_queue, &cmd, ...);
        switch (cmd.type) {
            case CMD_XYZ:
                ...
        }
    }
}
```



```
        xMessageBufferSend(tx_msg_buffer , ...);  
        break;  
    }  
    ...  
}  
}
```

2.6. Aplicación de Línea de Comandos

Para facilitar la interacción con el dispositivo y el procesamiento de las mediciones se desarrolló una aplicación de línea de comandos (CLI) en Python. La aplicación se comunica con el dispositivo a través de USB utilizando la librería *pyserial*, que encapsula el acceso al puerto serie. La misma se organiza en tres módulos principales:

- `serialpal.py`
- `cli.py`
- `json_exporter.py`

Adicionalmente, se desarrollaron scripts auxiliares para el procesamiento y visualización de datos.

2.6.1. Módulo `serialpal.py`

Este módulo actúa como capa de adaptación entre la interfaz de usuario y el dispositivo físico, encapsulando toda la lógica de comunicación serial. Para acceder al puerto serie del sistema operativo, se utilizó la librería *pyserial*, que provee una interfaz uniforme independientemente del sistema operativo del host.

La clase principal `SerialPal` provee métodos para listar los puertos disponibles del sistema, conectarse a uno de ellos, y enviar y recibir mensajes siguiendo el protocolo de comunicación descrito en la sección anterior.

El método `send()` construye y envía un comando con el formato `&<cmd>,<arg1>,<arg2>*`, mientras que `receive()` lee la respuesta del dispositivo interpretando primero el encabezado para determinar el tipo y la longitud del payload, y luego leyendo dicho payload de manera completa. Las respuestas recibidas se encapsulan en objetos de la clase `Response`, que almacena el tipo de respuesta (`TEXT` o `DATA`), el payload en formato binario, y su longitud en bytes.

El método `decode()` convierte el payload al tipo de dato correspondiente: en el caso de respuestas de texto, se decodifica como una cadena ASCII; en



el caso de datos binarios, se desempaquetan como una tupla de enteros sin signo de 8 o 16 bits, según se indique.

2.6.2. Módulo `cli.py`

Este módulo implementa la interfaz interactiva con el usuario, extendiendo la clase `cmd.Cmd` de la biblioteca estándar de Python. Esto permite ofrecer un intérprete de comandos con funcionalidades como autocompletado y un sistema de ayuda integrado.

Cada comando disponible en la interfaz se implementa como un método con el prefijo `do_`. Los comandos más relevantes son los siguientes:

- **ports**: lista los puertos serie disponibles en el sistema.
- **conn <puerto>**: establece la conexión con el dispositivo en el puerto indicado.
- **ping**: envía un comando de prueba al dispositivo para verificar la comunicación.
- **adc**: solicita al dispositivo una serie de muestras del ADC y calcula su media y desviación estándar.
- **set_led <índice> <rgb>**: configura el color de un LED del dispositivo mediante un código RGB de 24 bits.
- **sweep <canal>**: ejecuta el procedimiento de calibración para el canal de color indicado (R, G o B), descrito en la Apartado 2.3.
- **measure**: realiza una medición de color completa utilizando los parámetros del modelo LDR previamente calculados, y muestra el resultado como código RGB.

El comando **sweep** recorre un conjunto de intensidades con espaciado logarítmico en el rango 0 a 255, configurando el LED correspondiente en cada paso y registrando la respuesta del ADC. Durante el barrido se controla automáticamente la saturación del circuito de acondicionamiento, ajustando el valor de R_f cuando una muestra se encuentra fuera de la región de funcionamiento definida. Al finalizar, las mediciones se exportan a un archivo JSON mediante el módulo `json_exporter.py`.

El comando **measure** carga los parámetros K_a y γ del archivo `LDR_params.json` generado durante la calibración, enciende cada canal del LED a su máxima intensidad de manera secuencial, y determina el valor de color para



cada canal a partir de la Ecuación (6). Al finalizar, se muestra el resultado como un código RGB de 24 bits, junto con una representación visual del color obtenido en la terminal.

2.6.3. Scripts Auxiliares

El script `fit_LDR_curves.py` implementa el procedimiento de mínimos cuadrados descrito en la Apartado 2.3, utilizando las librerías *pandas* y *numpy*. Para cada canal de color, lee el archivo JSON correspondiente, calcula la conductancia G a partir de las muestras del ADC mediante la Ecuación (5), y aplica la Ecuación (9) para obtener los parámetros K_a y γ de la Ecuación (6). Los resultados se almacenan en el archivo `LDR_params.json`, que luego es utilizado por la aplicación CLI para realizar las mediciones.

Por su parte, el script `plot_LDR_response.py` permite visualizar la respuesta teórica del circuito de acondicionamiento para los distintos valores de R_f disponibles, graficando la tensión de salida V_o en función de la resistencia del LDR. Esto resulta útil para verificar el rango de funcionamiento del circuito antes de realizar mediciones.

3. Resultados

Se comprobó el funcionamiento del dispositivo y del proceso de calibración. Durante las pruebas, se calibró el dispositivo sobre papel de impresora, cuya superficie blanca y opaca asegura una reflexión mayormente difusa, lo cual es ideal para el funcionamiento correcto del dispositivo. Tal como se describió en los apartados anteriores, se ejecutó el comando `measure` para cada canal, y luego el script `fit_LDR_curves.py` para obtener los parámetros del LDR. Los resultados obtenidos se muestran en el Listado 4.

Listado 4: Parametros obtenidos de la caracterización del LDR.

```
{
  "R": {
    "Ka": 0.011508412425650682,
    "gamma": 0.8527065806062964
  },
  "G": {
    "Ka": 0.0032507832523976554,
    "gamma": 0.8597395131749697
  },
  "B": {
    "Ka": 0.0030240133142661603,
```



```

    "gamma": 0.8283233614433873
  }
}

```

Como se puede observar, el parámetro γ es prácticamente idéntico para los tres colores. Además, se concluye que los resultados obtenidos son coherentes con la Figura 2, ya que el LDR presenta distintas sensibilidades para cada color, ya que cada canal presenta distintos valores de K_a .

Por último, se tomaron medidas sobre superficies de distintos colores y se compararon los resultados con los de una aplicación móvil para medir colores. La aplicación elegida fue *Color Picker - Pixel Color*, disponible en el App Store de iOS. El dispositivo utilizado fue un iPhone 16.

Se utilizó la hoja de calibración mostrada en la Figura 8, la cuál cuenta con parches de color verde, amarillo, celeste y rosado. Se utilizó la aplicación móvil para obtener un color de referencia, y se comparó con los resultados obtenidos con el dispositivo. Las Figuras 9 a 12 muestran una comparación de los resultados obtenidos para cada parche de color.

En base a los resultados obtenidos se concluye que se lograron los objetivos propuestos para este trabajo. Se logró diseñar e implementar un dispositivo de bajo costo que permite estimar el color de una superficie, cuyos resultados son comparables con los obtenidos utilizando una cámara de buena calidad de un dispositivo móvil. Además, gracias al procedimiento de calibración, se logró obtener una curva de respuesta aproximada del LDR, y en teoría permitiría obtener resultados similares con otros LDR.

- [1] *CD4066 Datasheet*. Texas Instruments. URL: <https://www.ti.com/lit/ds/symlink/cd4066b.pdf>.
- [2] *LDR Datasheet*. Sparkfun Electronics. URL: <https://cdn.sparkfun.com/datasheets/Sensors/LightImaging/SEN-09088.pdf>.
- [3] Raspberry Pi. *Pico C SDK Documentation*. 2026. URL: <https://www.raspberrypi.com/documentation/pico-sdk/>.
- [4] *RP2040 Datasheet*. Raspberry Pi. 2026. URL: <https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf?disposition=inline>.
- [5] *TinyUSB Repository*. 2026. URL: <https://github.com/hathach/tinyusb>.

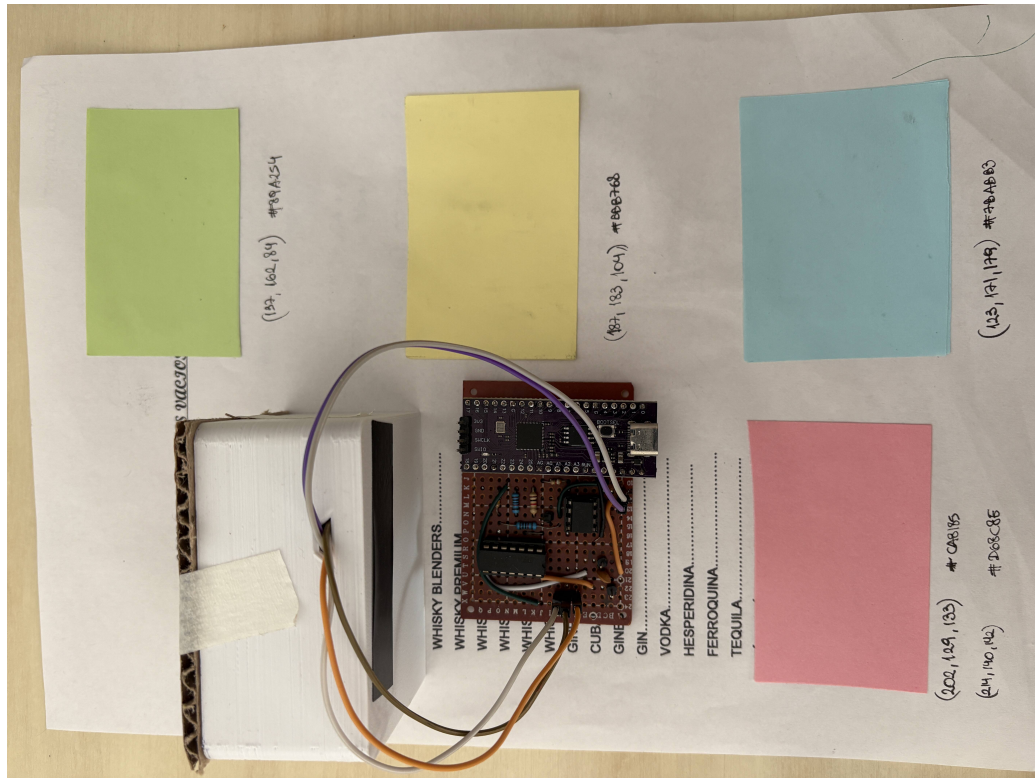


Figura 8: Hoja de calibración utilizada.

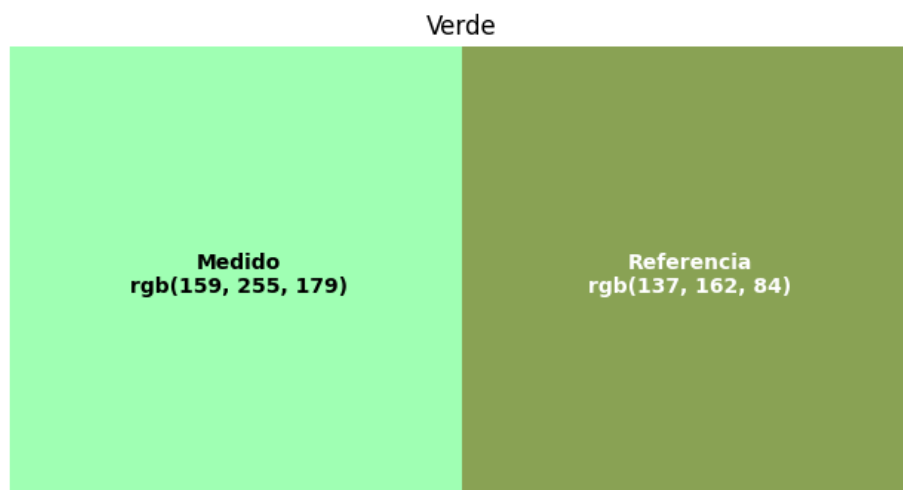


Figura 9: Comparación de resultados para la superficie verde.

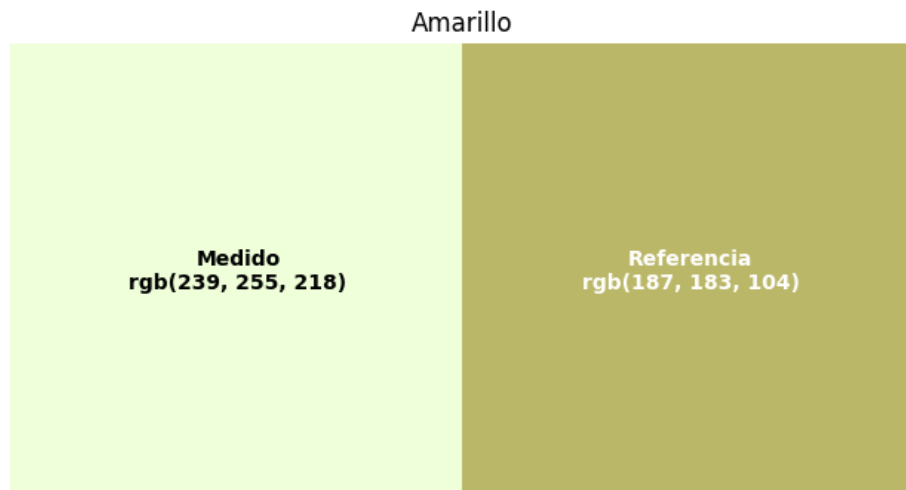


Figura 10: Comparación de resultados para la superficie amarilla.

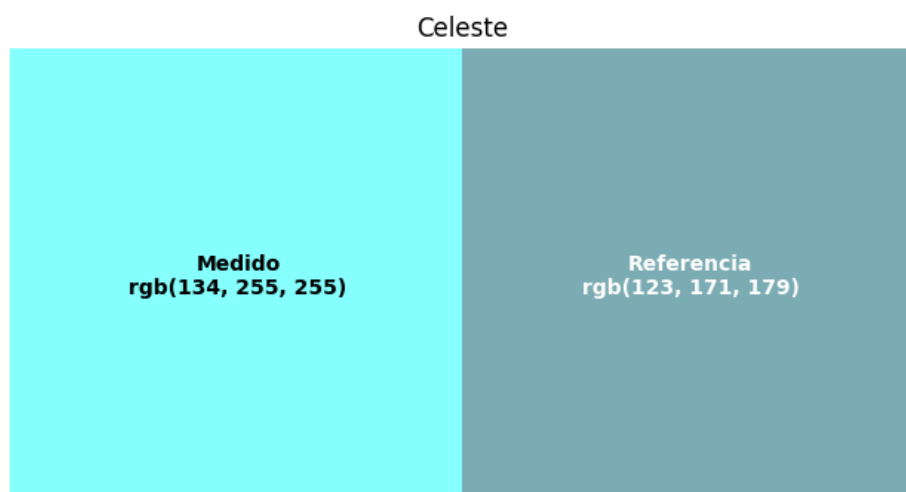


Figura 11: Comparación de resultados para la superficie celeste.

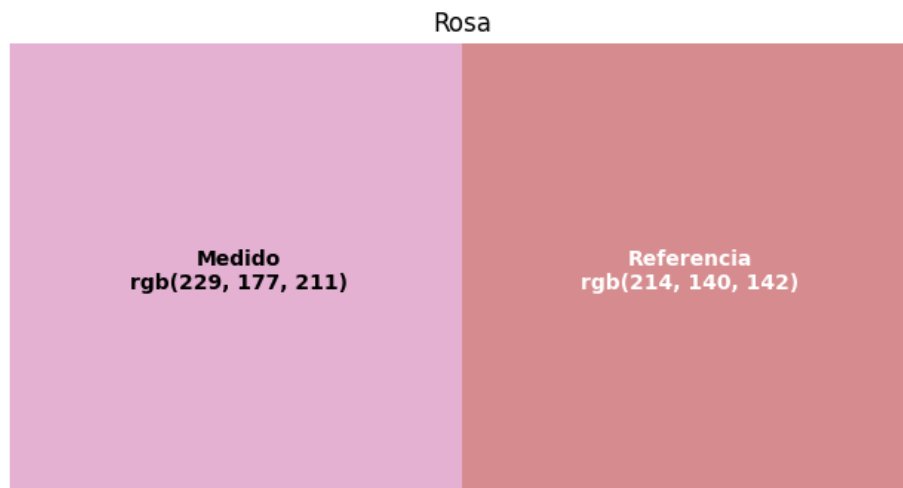


Figura 12: Comparación de resultados para la superficie rosa.